

Deep Learning

Vazgen Mikayelyan

May 15, 2021



Outline

- 1 Softmax Classifier
- 2 Stochastic Gradient Descent
- 3 Back-Propagation
- 4 Data Normalization

Softmax Classifier

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}^m$ (one-hot vectors) be our training data.

Softmax Classifier

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}^m$ (one-hot vectors) be our training data.
Consider the function

$$f(x) = \left(\frac{e^{w_1^T x + b_1}}{\sum_{i=1}^m e^{w_i^T x + b_i}}, \dots, \frac{e^{w_m^T x + b_m}}{\sum_{i=1}^m e^{w_i^T x + b_i}} \right)$$

Softmax Classifier

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}^m$ (one-hot vectors) be our training data. Consider the function

$$f(x) = \left(\frac{e^{w_1^T x + b_1}}{\sum_{i=1}^m e^{w_i^T x + b_i}}, \dots, \frac{e^{w_m^T x + b_m}}{\sum_{i=1}^m e^{w_i^T x + b_i}} \right)$$

Our aim is to find parameters $(b_i, w_i)_{i=1}^m$ such that

$$f(x_i) \approx y_i, i = 1, \dots, n.$$

Softmax Classifier

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}^m$ (one-hot vectors) be our training data. Consider the function

$$f(x) = \left(\frac{e^{w_1^T x + b_1}}{\sum_{i=1}^m e^{w_i^T x + b_i}}, \dots, \frac{e^{w_m^T x + b_m}}{\sum_{i=1}^m e^{w_i^T x + b_i}} \right)$$

Our aim is to find parameters $(b_i, w_i)_{i=1}^m$ such that

$$f(x_i) \approx y_i, i = 1, \dots, n.$$

We choose cross entropy distance as our loss function:

$$\frac{1}{n} \sum_{i=1}^n \left(-y_i^T \log f(x_i) \right).$$

- 1 Can you represent this model as a neural network?

Questions

- 1 Can you represent this model as a neural network?
- 2 Can we use the function sigmoid in this case?

- 1 Can you represent this model as a neural network?
- 2 Can we use the function sigmoid in this case?
- 3 What to do in the case of multi-label classification?

Outline

- 1 Softmax Classifier
- 2 Stochastic Gradient Descent
- 3 Back-Propagation
- 4 Data Normalization

Stochastic Gradient Descent

Let L be a loss function that we know:

Stochastic Gradient Descent

Let L be a loss function that we know:

$$L(w) = \frac{1}{n} \sum_{i=1}^n (f_w(x_i) - y_i)^2,$$

$$L(w) = \frac{1}{n} \sum_{i=1}^n (-y_i \log f_w(x_i) - (1 - y_i) \log (1 - f_w(x_i))),$$

$$L(w) = \frac{1}{n} \sum_{i=1}^n \left(-y_i^T \log f_w(x_i) \right).$$

Stochastic Gradient Descent

Let L be a loss function that we know:

$$L(w) = \frac{1}{n} \sum_{i=1}^n (f_w(x_i) - y_i)^2,$$

$$L(w) = \frac{1}{n} \sum_{i=1}^n (-y_i \log f_w(x_i) - (1 - y_i) \log (1 - f_w(x_i))),$$

$$L(w) = \frac{1}{n} \sum_{i=1}^n \left(-y_i^T \log f_w(x_i) \right).$$

Do you see problems in finding minimum of these functions using GD?

Note that in each case we can represent the loss function by the following form:

$$L(w) = \frac{1}{n} \sum_{i=1}^n L_i(w).$$

Note that in each case we can represent the loss function by the following form:

$$L(w) = \frac{1}{n} \sum_{i=1}^n L_i(w).$$

SGD algorithm is the following:

- Choose an initial vector of parameters w and learning rate α .

Note that in each case we can represent the loss function by the following form:

$$L(w) = \frac{1}{n} \sum_{i=1}^n L_i(w).$$

SGD algorithm is the following:

- Choose an initial vector of parameters w and learning rate α .
- Repeat until an approximate minimum is obtained.

Note that in each case we can represent the loss function by the following form:

$$L(w) = \frac{1}{n} \sum_{i=1}^n L_i(w).$$

SGD algorithm is the following:

- Choose an initial vector of parameters w and learning rate α .
- Repeat until an approximate minimum is obtained.
 - Randomly shuffle examples in the training set.

Note that in each case we can represent the loss function by the following form:

$$L(w) = \frac{1}{n} \sum_{i=1}^n L_i(w).$$

SGD algorithm is the following:

- Choose an initial vector of parameters w and learning rate α .
- Repeat until an approximate minimum is obtained.
 - Randomly shuffle examples in the training set.
 - For $i = 1, 2, \dots, n$, do $w \leftarrow w - \alpha \nabla L_i(w)$.

Stochastic Gradient Descent

Note that in each case we can represent the loss function by the following form:

$$L(w) = \frac{1}{n} \sum_{i=1}^n L_i(w).$$

SGD algorithm is the following:

- Choose an initial vector of parameters w and learning rate α .
- Repeat until an approximate minimum is obtained.
 - Randomly shuffle examples in the training set.
 - For $i = 1, 2, \dots, n$, do $w \leftarrow w - \alpha \nabla L_i(w)$.

Do you see problems in this optimization method?

Mini-Batch Gradient Descent

MBGD algorithm is the following:

- Choose an initial vector of parameters w , learning rate α and batch size B .

Mini-Batch Gradient Descent

MBGD algorithm is the following:

- Choose an initial vector of parameters w , learning rate α and batch size B .
- Repeat until an approximate minimum is obtained.

Mini-Batch Gradient Descent

MBGD algorithm is the following:

- Choose an initial vector of parameters w , learning rate α and batch size B .
- Repeat until an approximate minimum is obtained.
 - Randomly shuffle examples in the training set.

Mini-Batch Gradient Descent

MBGD algorithm is the following:

- Choose an initial vector of parameters w , learning rate α and batch size B .
- Repeat until an approximate minimum is obtained.
 - Randomly shuffle examples in the training set.
 - For $i = 1, 2, \dots, \lceil \frac{n}{B} \rceil$, do

$$w \leftarrow w - \alpha \nabla \frac{1}{B} \sum_{k=(i-1) \cdot B + 1}^{i \cdot B} L_k(w).$$

Outline

- 1 Softmax Classifier
- 2 Stochastic Gradient Descent
- 3 Back-Propagation**
- 4 Data Normalization

Question: How to calculate the derivative of the function $\sin x^2$?

Question: How to calculate the derivative of the function $\sin x^2$?

Theorem 1

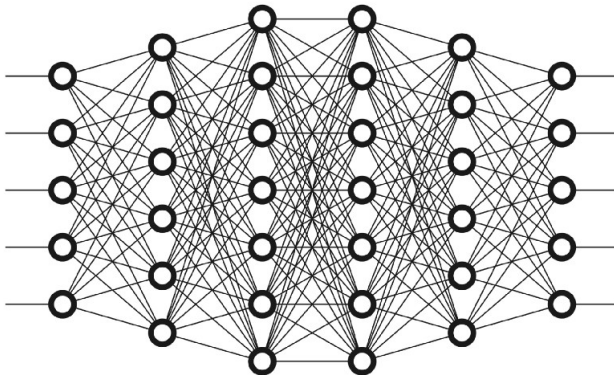
Given n functions $f_1, \dots, f_n$ with the composite function

$$f = f_1 \circ (f_2 \circ \dots (f_{n-1} \circ f_n)),$$

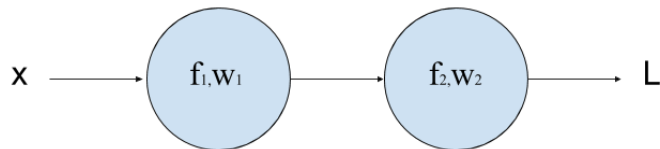
if each function f_i is differentiable at its immediate input, then the composite function is also differentiable by the repeated application of Chain Rule, where the derivative is

$$\frac{df}{dx} = \frac{df_1}{df_2} \frac{df_2}{df_3} \dots \frac{df_n}{dx}.$$

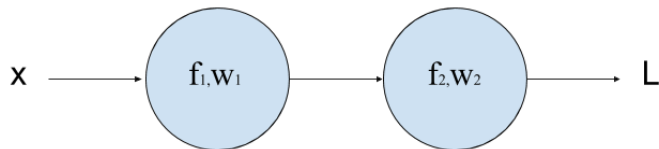
Back-Propagation



Back-Propagation



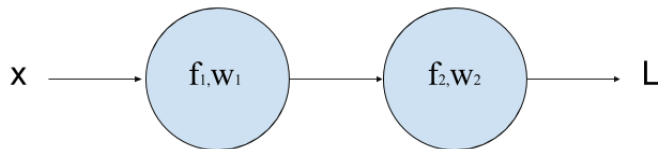
Back-Propagation



In this case we have the following function

$$L(w_1, w_2) = (f_2(w_2 f_1(w_1 x)) - y)^2$$

Back-Propagation

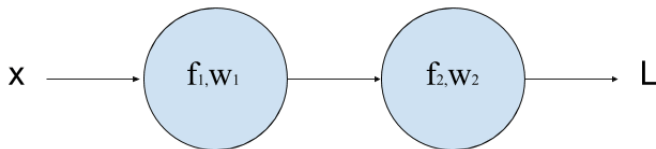


In this case we have the following function

$$L(w_1, w_2) = (f_2(w_2 f_1(w_1 x)) - y)^2$$

We have to calculate the derivatives $\frac{\partial L}{\partial w_1}$ and $\frac{\partial L}{\partial w_2}$:

Back-Propagation



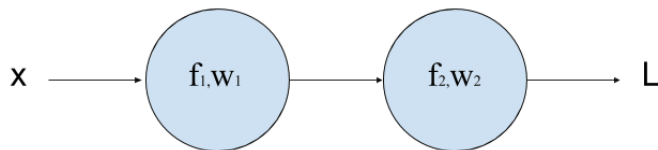
In this case we have the following function

$$L(w_1, w_2) = (f_2(w_2 f_1(w_1 x)) - y)^2$$

We have to calculate the derivatives $\frac{\partial L}{\partial w_1}$ and $\frac{\partial L}{\partial w_2}$:

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial f_2} \frac{\partial f_2}{\partial (w_2 f_1)} \frac{\partial (w_2 f_1)}{\partial w_2},$$

Back-Propagation



In this case we have the following function

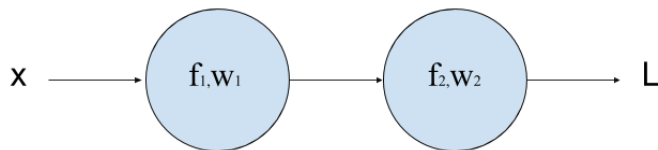
$$L(w_1, w_2) = (f_2(w_2 f_1(w_1 x)) - y)^2$$

We have to calculate the derivatives $\frac{\partial L}{\partial w_1}$ and $\frac{\partial L}{\partial w_2}$:

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial f_2} \frac{\partial f_2}{\partial (w_2 f_1)} \frac{\partial (w_2 f_1)}{\partial w_2},$$

$$\frac{\partial L}{\partial w_1} =$$

Back-Propagation



In this case we have the following function

$$L(w_1, w_2) = (f_2(w_2 f_1(w_1 x)) - y)^2$$

We have to calculate the derivatives $\frac{\partial L}{\partial w_1}$ and $\frac{\partial L}{\partial w_2}$:

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial f_2} \frac{\partial f_2}{\partial (w_2 f_1)} \frac{\partial (w_2 f_1)}{\partial w_2},$$

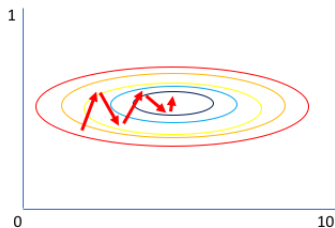
$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial f_2} \frac{\partial f_2}{\partial (w_2 f_1)} \frac{\partial (w_2 f_1)}{\partial f_1} \frac{\partial (f_1)}{\partial (w_1 x)} \frac{\partial (w_1 x)}{\partial w_1}.$$

Outline

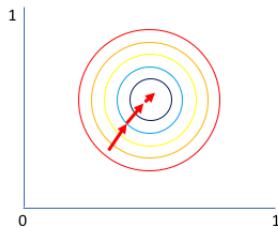
- 1 Softmax Classifier
- 2 Stochastic Gradient Descent
- 3 Back-Propagation
- 4 Data Normalization**

Data Normalization

Why normalize?



Gradient of larger parameter
dominates the update



Both parameters can be
updated in equal proportions

Standard Normalization

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}^m$ be our training data:

Standard Normalization

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}^m$ be our training data:

$$X = \begin{bmatrix} x_1^1 & x_1^2 & \dots & x_1^k \\ x_2^1 & x_2^2 & \dots & x_2^k \\ \vdots & \vdots & \vdots & \vdots \\ x_n^1 & x_n^2 & \dots & x_n^k \end{bmatrix}.$$

Standard Normalization

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}^m$ be our training data:

$$X = \begin{bmatrix} x_1^1 & x_1^2 & \dots & x_1^k \\ x_2^1 & x_2^2 & \dots & x_2^k \\ \vdots & \vdots & \vdots & \vdots \\ x_n^1 & x_n^2 & \dots & x_n^k \end{bmatrix}.$$

Denote $\mu^j = \frac{1}{n} \sum_{i=1}^n x_i^j$, $\sigma^j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i^j - \mu^j)^2}$

Standard Normalization

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}^m$ be our training data:

$$X = \begin{bmatrix} x_1^1 & x_1^2 & \dots & x_1^k \\ x_2^1 & x_2^2 & \dots & x_2^k \\ \vdots & \vdots & \vdots & \vdots \\ x_n^1 & x_n^2 & \dots & x_n^k \end{bmatrix}.$$

Denote $\mu^j = \frac{1}{n} \sum_{i=1}^n x_i^j$, $\sigma^j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i^j - \mu^j)^2}$ and let

$$z_i^j = \frac{x_i^j - \mu^j}{\sigma^j}.$$

Standard Normalization

Let $(x_i, y_i)_{i=1}^n$, $x_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}^m$ be our training data:

$$X = \begin{bmatrix} x_1^1 & x_1^2 & \dots & x_1^k \\ x_2^1 & x_2^2 & \dots & x_2^k \\ \vdots & \vdots & \vdots & \vdots \\ x_n^1 & x_n^2 & \dots & x_n^k \end{bmatrix}.$$

Denote $\mu^j = \frac{1}{n} \sum_{i=1}^n x_i^j$, $\sigma^j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i^j - \mu^j)^2}$ and let

$$z_i^j = \frac{x_i^j - \mu^j}{\sigma^j}.$$

Our new data will be $(z_i, y_i)_{i=1}^n$, $z_i \in \mathbb{R}^k$, $y_i \in \mathbb{R}^m$.